Porting the Excel 2000 “Dev Hunter” Easter Egg

In *Paged Out!* #6 I described **Dev Hunter**, the hidden car-shooter easter egg buried in Excel 2000’s `MSOWC.DLL`, and a loader that runs it outside Excel. You drive a highway hunting the Excel developers themselves, whose names scroll past painted on the road as mock movie credits (“Stunts: Additional”) between taunts like “Pivot! Pivot! Pivot!” and “You will respect the rectangles.” But the loader still needs the original DLL, and Excel 2000 with its DirectDraw path only gets harder to run every year. I wanted to free the game completely: reimplement it as a small standalone program — no Excel, no DirectDraw, no DLL — that anyone can build, read, and play, so the easter egg outlives its host. This time I didn’t write it myself. I had an AI agent (**Claude Code**, Anthropic’s agentic coding tool) do the entire job — reverse-engineer `MSOWC.DLL` and translate the game into a standalone C++ program — while I steered.

Working from Binary Ninja’s decompilation, the AI rebuilt the game from the recovered palette, decrypted developer text, RNG, and projection constants, and translated the DLL’s real `render` and rasterizer routines into C. It built and ran, but the rendering was clearly off (Fig. 1, left): developer names spilling off the tarmac, the road at the wrong scale, cars reduced to blocks. The integer and pointer code came across faithfully; the geometry, which runs entirely on the x87 FPU stack, is the kind of code decompilers reconstruct imperfectly — so the translation was subtly wrong, and re-reading the listing wasn’t fixing it.

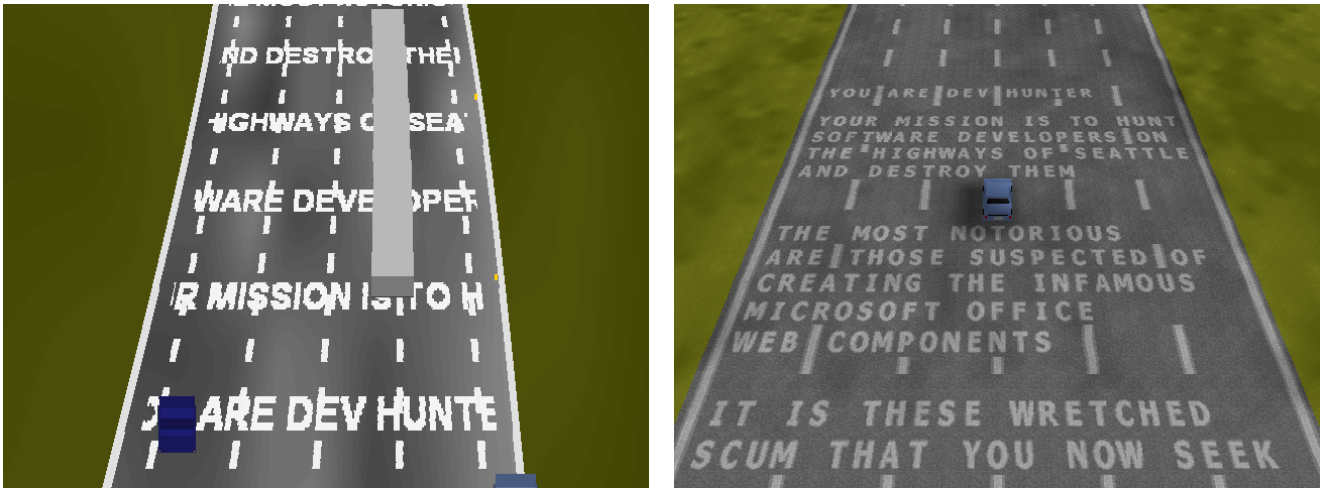


Figure 1: The same intro screen: the initial standalone build (left) vs. the final debugger-verified port (right).

At this point I decided to intervene: I pointed the AI at the debugger, to observe the program’s real state instead of reading the code statically. The AI turned that into an automated oracle — it scripted Binary Ninja’s headless debugger to launch the game, break right before the DirectDraw `Unlock`, and read the live 8-bit framebuffer at `*(this+0x34)` (pitch `*(this+0x20)`) plus the entire ∼`0x9a96c`-byte game object, then diffed that ground truth against its own port frame by frame and checked the physics by comparing the object before and after a single `tick`. Each mismatch became an exact, usually one-line fix (a sheared projection edge, a dropped floating-point value, a bad texture mask).

The port is now **byte-identical** to the original across a full day/night cycle. Getting a clean bit-for-bit comparison took two tricks: the game’s `init` — terrain, textures, the 3-D letter atlas — is deterministic (RNG seed 0), so rather than re-derive its heavy x87 setup the port loads a snapshot of `init`’s exact output and relocates its pointers; and because the game times itself off the wall clock, headless runs only became reproducible once a fixed virtual clock was wired in — then SHA-256 over the frame dumps could prove equivalence. The lesson: the AI could carry the whole reverse-engineering port, but static decompilation only got it most of the way — the idea that closed the gap was a human one, giving it a debugger to observe ground truth.

Source, build instructions, and a playable Win32 release:
`https://github.com/xusheng6/excel2000-devhunter-standalone`